

Design Specification: PARD Programming Language

Erdinç Daşkın

June 2, 2026

Contents

1	Introduction & Philosophy	2
2	Lexical Grammar	2
2.1	Variables	2
2.1.1	Variables and their types	2
2.1.2	Variable definition	2
2.1.3	Simple usage of variables	2
2.1.4	Converting variables to other types	2
2.1.5	Array definition	3
2.1.6	Accessing values in an array	3
2.1.7	Constants	3
2.2	Procedures	3
2.2.1	Routines	3
2.2.2	Subroutines	3
2.2.3	Functions	4
2.2.4	Passing parameters by value	4
3	Compiler Architecture	4

1 Introduction & Philosophy

PARD is a programming language designed around game design and development. It leverages FORTRAN's fast math and C interoperability. The name PARD is an abbreviation of Punching Card, which is an homage to the early days of FORTRAN.

2 Lexical Grammar

2.1 Variables

2.1.1 Variables and their types

Variables, just like most other programming languages, are a significant part of PARD. There's 4 basic variable types:

- **int**: A standard integer type for counts, loop counters, and IDs.
- **float**: A floating-point number for positions, physics calculations, and deltatime.
- **bool**: A boolean value (**true** or **false**) for state flags.
- **string[N]**: A fixed-length, ultra-fast character array of size N .

As you can see, strings are not dynamically sized - this is not only for easy operation in fortran, but it also introduces a good performance boost.

2.1.2 Variable definition

You can easily define a variable in the following way:

```
int myInt = 10
float myFloat = 10.5
bool myBool = true
string[6] myString = 'Hello!'
```

2.1.3 Simple usage of variables

You can easily use a variable in multiple ways, here are some:

```
int x = 2
int y = 3
int z = x + y
float f = 2.5
float g = 0.5
float h = f / g
```

You cannot mix types: by this I mean you can't do the following:

```
int x = 2
float y = 10.5
float z = x + y
```

This will raise an error, as not all types used together are of the same type.

2.1.4 Converting variables to other types

In the previous example, we had this:

```
int x = 2
float y = 10.5
float z = x + y
```

But what if we want to combine different types? Well, you can use simple parsing functions:

```
float z = x + y.tofloat
```

There exists tofloat, toint and toString.

2.1.5 Array definition

Okay, how do we define arrays?

Before the type, put (n) where n is the length of said array.

```
(2)int myInt = [10, 5]
```

```
(3)float myFloat = [10.5, 3.14, 5.5]
```

```
(3)bool myBool = [true,false,false]
```

```
(2)string[6] myString = ['Hello ', 'World!']
```

2.1.6 Accessing values in an array

To get a part of an array, use an index. PARD is 1-index based, e.g. the first item is index 1, the second is index 2 etc.

```
(2)int myIntArray = [10, 5]
```

```
int myInt = myIntArray[2]
```

In this example, myInt will be 5.

2.1.7 Constants

One can easily define constants using the const keyword.

```
const (2)int myIntArray = [10, 5]
```

```
const float y = 10.5
```

2.2 Procedures

One may ask, what is the difference between procedures and functions in PARD. Well, the difference is, that functions always return a value, while not all procedures have to.

All procedure parameters are passed by reference by default.

There's 3 types of procedures:

2.2.1 Routines

Routines are procedures that don't return any value. They are defined by the rout keyword and require to have at least 1 parameter.

```
rout myRoute(int x, int y){  
  x = y  
}
```

Keep in mind, routine parameters are by default passed by reference, and in this case, the original x passed will be modified.

2.2.2 Subroutines

Subroutines are routines that don't take any parameters. They are defined by the sub keyword.

```
sub mySub {  
  pass  
}
```

the pass keyword simply does nothing.

2.2.3 Functions

Functions are like routes that return a value. They are defined by the keyword `fn`. They may or may not have parameters.

```
fn myFunc(int x, int y){  
  return x  
}
```

2.2.4 Passing parameters by value

If you do not wish a routine/function to affect variables passed into it, you may use the value indicator `&`:

```
rout myRoute(&int x, int y){  
  x = y  
}
```

In this case, if a variable is passed as the parameter `x`, it won't change to `y` globally.

3 Compiler Architecture